3B2 FPGA Programming Lab Report

Liu Zihe[1]*

[1]University of Cambridge

Abstract

This report documents the design, implementation and testing of a traffic light controller on an Altera Cyclone V FPGA. A three-state finite state machine controlling a pedestrian crossing is described in Verilog, compiled, and configured onto a DE1-SoC board via JTAG, and is improved with a vehicle lockout.

1 Traffic Light Controller Design

We design a controller for a pedestrian light-controlled crossing, satisfying the following requirements:

1. *The initial state is vehicle GREEN and pedestrian RED.*
2. *State changes occur only on demand: a pedestrian request or a reset.*
3. *After a request, the sequence is: vehicle YELLOW for 5 s, then vehicle RED with pedestrian GREEN for 10 s, then back to the initial state.*
4. *An asynchronous reset returns to the initial state from any state.*

These requirements are captured by a three-state FSM shown in Figure 1. State `G` is the idle state with vehicle GREEN and pedestrian RED. A pedestrian `request` triggers a transition to state `Y` (vehicle YELLOW), which serves as a warning to vehicles. After a 5 s timeout the FSM enters state `R` (vehicle RED, pedestrian GREEN), allowing pedestrians to cross. A 10 s timeout then returns the FSM to state `G`. An asynchronous active-low `reset` forces the FSM back to state `G` from any state.

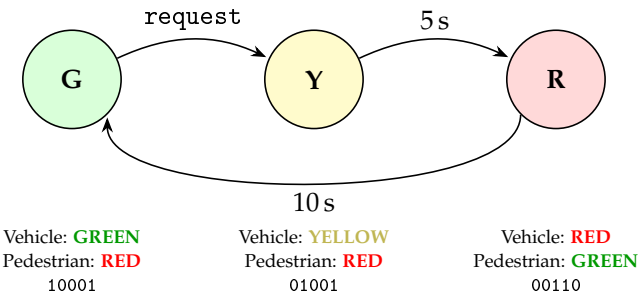

Figure 1: State diagram for the basic TLC FSM. `reset` (active LOW, asynchronous) returns to state `G` from any state.

The outputs of each state and the transitions to the next states are summarised in Table 1. Each state activates exactly one vehicle light and one pedestrian light.

Table 1: State table for the traffic light controller.

	Next State				Outputs				
State	`reset`	`request`	5 s	10 s	`veh_G`	`veh_Y`	`veh_R`	`ped_G`	`ped_R`
`G`	`G`	`Y`	–	–	1	0	0	0	1
`Y`	`G`	–	`R`	–	0	1	0	0	1
`R`	`G`	–	–	`G`	0	0	1	1	0

2 Verilog Implementation

Listing 1: Basic traffic light controller.

```
1 module tlc (
2     input  wire        clk,
```

*CRSID: zl559. Date: 20th February 2026

```
    input  wire        request,
    input  wire        reset,
    output reg  [4:0] \output   // {veh_G, veh_Y, veh_R, ped_G, ped_R}
);

  localparam G = 2'd0, Y = 2'd1, R = 2'd2;

  reg [ 1:0] state;
  reg [28:0] count;

  // Sequential: state transitions & timer
  always @(posedge clk or negedge reset) begin
    if (!reset) begin
      state <= G;
      count <= 29'd0;
    end else begin
      case (state)
        G: begin
          if (request == 1'b0) begin
            state <= Y;
            count <= 29'd0;
          end
        end
        Y: begin
          if (count == 29'd250_000_000) begin // 5s @ 50 MHz
            state <= R;
            count <= 29'd0;
          end else count <= count + 29'd1;
        end
        R: begin
          if (count == 29'd500_000_000) begin // 10s @ 50 MHz
            state <= G;
            count <= 29'd0;
          end else count <= count + 29'd1;
        end
        default: begin
          state <= G;
          count <= 29'd0;
        end
      endcase
    end
  end

  // Combinational: output mapping
  always @(*) begin
    case (state)
      G:       \output = 5'b10001;
      Y:       \output = 5'b01001;
      R:       \output = 5'b00110;
      default: \output = 5'b10001;
    endcase
  end

endmodule
```

The Verilog code in Listing 1 implements the FSM from Section 1. Key design choices made are:

- **Clock division.** The DE1-SoC provides a 50 MHz clock, but the TLC requires delays on the order of seconds. Rather than a separate clock divider module, a 29-bit counter (`count`) is used directly.
- **State encoding.** The three states are encoded with 2 bits (`G`=0, `Y`=1, `R`=2). A `default` case catches the unused encoding (3) and returns to state `G`.
- **Active-low inputs.** The DE1-SoC push-buttons output 0 when pressed. The reset is handled via `negedge reset` in the sensitivity list for asynchronous behaviour, and `request` is checked against `1'b0`.
- **Output mapping.** A combinational `always @(*)` block maps each state to a 5-bit output vector `[4:0]` = `{veh_G, veh_Y, veh_R, ped_G, ped_R}`.

3 Compilation Results

The design was compiled in Quartus Prime 25.1 Lite Edition targeting the Cyclone V 5CSEMA5F31C6 device. The full Compiler Flow Summary screenshot is in Appendix A; key figures are in Table 2.

Table 2: Compiler Flow Summary.

Resource	Count
Logic utilisation (ALMs)	27
Total registers	32
Total pins	8

The 32 registers break down as follows: the 2-bit `state` register, the 29-bit `count` register, and 1 synchroniser flip-flop for the `request` input. The 27 Adaptive Logic Modules (ALMs) house the counter increment, ad state transition logic. The 8 pins match the 3 inputs (`clk`, `request`, `reset`) and 5 outputs (`output[4:0]`).

4 Pin Assignment

During initial compilation, Quartus is free to assign design signals to any FPGA pin. However, the DE1-SoC board has hardwired connections between specific FPGA pins and on-board components (LEDs, push-buttons, clock oscillator), so pin assignments must be set manually via the Assignment Editor to match. The assignments used are shown in Table 3. The design was recompiled after assigning pins so I/O is fitted correctly.

One limitation of the DE1-SoC is that all ten on-board user LEDs (LEDR0–LEDR9) are physically red. The traffic light colours (green, yellow, red) are therefore represented by LED position rather than colour: for example, LEDR8 represents vehicle GREEN even though the physical LED is red. Every output was placed at a different even integer position.

Table 3: Pin assignments for the DE1-SoC board.

Signal	Pin	Component
`clk`	`PIN_AF14`	50 MHz Clock
`request`	`PIN_AA15`	KEY1 (Push-button)
`reset`	`PIN_AA14`	KEY0 (Push-button)
`output[0]`	`PIN_V16`	LEDR0 (ped RED)
`output[1]`	`PIN_V17`	LEDR2 (ped GREEN)
`output[2]`	`PIN_W17`	LEDR4 (veh RED)
`output[3]`	`PIN_Y19`	LEDR6 (veh YELLOW)
`output[4]`	`PIN_W21`	LEDR8 (veh GREEN)

5 Testing

After programming the FPGA via JTAG, the circuit was tested on-board by exercising each transition in Table 1 and verifying the LED outputs. The results are summarised in Table 4.

Table 4: Board testing results.

Initial State	Input Applied	Expected Next State	Pass/Fail
`G`	`request`	`Y` (5 s timer)	Pass
`Y`	5 s timeout	`R` (10 s timer)	Pass
`R`	10 s timeout	`G`	Pass
Any	`reset`	`G`	Pass

All transitions matched the expected behaviour. Pressing KEY1 (`request`) in state `G` immediately turned on LEDR6 (vehicle YELLOW); after approximately 5 s, LEDR4 (vehicle RED) and LEDR2 (pedestrian GREEN) turned on; after a further 10 s the system returned to LEDR8 (vehicle GREEN) and LEDR0

(pedestrian RED). Pressing KEY0 (`reset`) at any point during the sequence returned the circuit to state `G` immediately, confirming the asynchronous reset. A photo of the board in state `G` is shown in Figure 4.

A functional simulation testbench (Listing 3 in Appendix A) was also used to verify the design in Icarus Verilog prior to board programming, with all 13 assertions passing.

6 Improved Circuit

In the basic TLC, a pedestrian who holds the `request` button continuously causes the FSM to cycle `G`→`Y`→`R`→`G`→`Y`→ $\cdots$ with no gap, permanently blocking the vehicle lane on RED. To prevent this, a fourth state `L` (lockout) is added: after completing state `R`, the FSM enters `L` which holds vehicle GREEN for 10 s while ignoring any `request`. Only after the lockout expires does the FSM return to `G`, where requests are accepted again. The updated state diagram is shown in Figure 2.

6.1 Updated State Machine

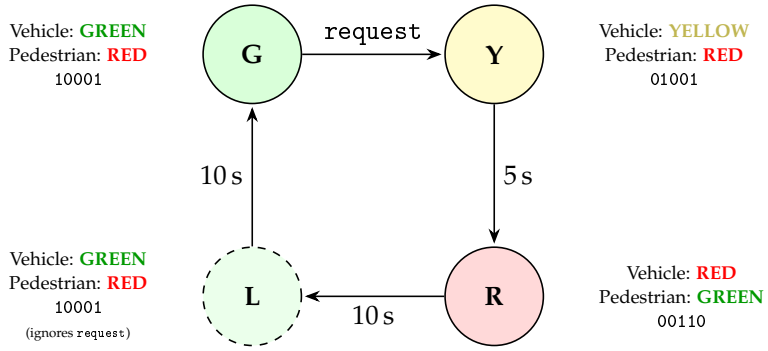

Figure 2: Improved TLC with lockout state `L`. After state `R`, the FSM holds vehicle GREEN for 10 s in `L`, ignoring requests. `reset` returns to `G` from any state.

The key change is that state `R` now transitions to `L` (not `G`), and `L` transitions to `G` after a 10 s timeout. State `L` has the same output as `G` (`10001`: vehicle GREEN, pedestrian RED) but does not respond to `request`. All four states fit in the existing 2-bit encoding (`L`=3), so no additional register bits are needed.

6.2 Verilog Implementation

Listing 2: Improved TLC with lockout.

```
// Improved Traffic Light Controller with lockout
// Adds state L: 10s vehicle GREEN lockout after pedestrian crossing

module tlc_lock (
    input  wire       clk,
    input  wire       request,
    input  wire       reset,
    output reg  [4:0] \output   // {veh_G, veh_Y, veh_R, ped_G, ped_R}
);

  localparam G = 2'd0, Y = 2'd1, R = 2'd2, L = 2'd3;

  reg [ 1:0] state;
  reg [28:0] count;

  // Sequential: state transitions & timer
  always @(posedge clk or negedge reset) begin
    if (!reset) begin
      state <= G;
      count <= 29'd0;
    end else begin
      case (state)
        G: begin
          if (request == 1'b0) begin
            state <= Y;
            count <= 29'd0;
```

```
27            end
28          end
29          Y: begin
30            if (count == 29'd250_000_000) begin // 5s @ 50 MHz
31              state <= R;
32              count <= 29'd0;
33            end else count <= count + 29'd1;
34          end
35          R: begin
36            if (count == 29'd500_000_000) begin // 10s @ 50 MHz
37              state <= L;
38              count <= 29'd0;
39            end else count <= count + 29'd1;
40          end
41          L: begin // 10s lockout, ignores request
42            if (count == 29'd500_000_000) begin // 10s @ 50 MHz
43              state <= G;
44              count <= 29'd0;
45            end else count <= count + 29'd1;
46          end
47        endcase
48      end
49    end
50
51    // Combinational: output mapping
52    always @(*) begin
53      case (state)
54        G:        \output = 5'b10001;
55        Y:        \output = 5'b01001;
56        R:        \output = 5'b00110;
57        L:        \output = 5'b10001; // same as G but ignores request
58      endcase
59    end
60
61  endmodule
```

Compared to the basic design, the changes in Listing 2 are minimal: state `R` transitions to `L` instead of `G`, and a new `L` case reuses the same 10 s counter threshold. Since all four 2-bit encodings are now used, the `default` case is no longer needed.

6.3 Testing

The same test cases from previously were repeated for the improved design, all passing. An additional test verified the lockout: `request` was held low throughout state `L`, and the FSM correctly remained in `L` until the 10 s timeout expired before returning to `G`. The improved circuit was also compiled, programmed onto the board via JTAG, and tested on-hardware with the same results.

7 Conclusions

Both the basic and improved TLC designs were successfully implemented on the Cyclone V FPGA, with all state transitions and timing verified on-board. The lockout state prevented continuous requests from blocking the vehicle lane, and fitted within the existing 2-bit encoding. Verilog behavioural entry made the improvement straightforward compared to a schematic redesign.

A Appendix

A.1 Compilation Report

Compilation Report - tlc

Flow Summary

<<Filter>>

Flow Status	Successful - Fri Feb 20 13:21:29 2026
Quartus Prime Version	25.1std.0 Build 1129 10/21/2025 SC Lite Edition
Revision Name	tlc
Top-level Entity Name	tlc
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	27 / 32,070 (< 1 %)
Total registers	32
Total pins	8 / 457 (2 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 3: Quartus Prime Compiler Flow Summary for the basic TLC design.

A.2 Board Photo

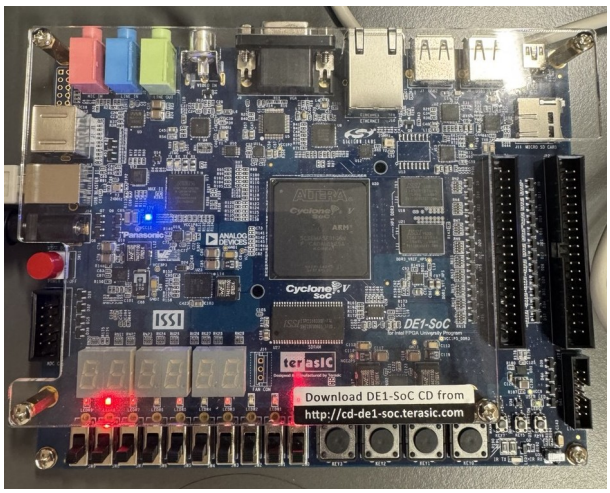

Figure 4: DE1-SoC board in state `G`: LEDR8 (vehicle GREEN) and LEDR0 (pedestrian RED) are lit.

A.3 Testbench

Listing 3: Testbench for the basic TLC.

```
// Testbench for basic Traffic Light Controller (tlc.v)
// Uses force/release on internal counter to skip long timer waits.
`timescale 1ns / 1ps

module tlc_tb;

  reg clk, request, reset;
  wire [4:0] out;

  tlc uut (.clk(clk), .request(request), .reset(reset), .\output (out));

  initial clk = 0;
  always #10 clk = ~clk; // 50 MHz
```

```
14
15    localparam G = 2'd0, Y = 2'd1, R = 2'd2;
16    localparam OUT_G = 5'b10001, OUT_Y = 5'b01001, OUT_R = 5'b00110;
17    localparam Y_THRESH = 29'd250_000_000, R_THRESH = 29'd500_000_000;
18
19    integer pass_count = 0, fail_count = 0;
20
21    task check(input [63:0] name, input cond);
22      begin
23        if (cond) begin $display("  PASS: %0s", name); pass_count = pass_count + 1; end
24        else      begin $display("  FAIL: %0s", name); fail_count = fail_count + 1; end
25      end
26    endtask
27
28    task wait_clks(input integer n);
29      integer i;
30      begin for (i = 0; i < n; i = i + 1) @(posedge clk); end
31    endtask
32
33    // Skip counter to near threshold, then let it trigger naturally
34    task skip_counter(input [28:0] thresh);
35      begin
36        force uut.count = thresh - 2;
37        wait_clks(1);
38        release uut.count;
39        wait_clks(5);
40      end
41    endtask
42
43    initial begin
44      $dumpfile("tlc_tb.vcd");
45      $dumpvars(0, tlc_tb);
46      request = 1; reset = 1;
47
48      // T1: Reset -> state G
49      $display("\n--- T1: Reset ---");
50      reset = 0; wait_clks(3);
51      check("state=G", uut.state == G);
52      check("out=10001", out == OUT_G);
53      reset = 1; wait_clks(2);
54
55      // T2: Full cycle G -> Y -> R -> G
56      $display("\n--- T2: Full cycle ---");
57      request = 0; wait_clks(2); request = 1;
58      check("G->Y", uut.state == Y);
59      check("out=01001", out == OUT_Y);
60      skip_counter(Y_THRESH);
61      check("Y->R", uut.state == R);
62      check("out=00110", out == OUT_R);
63      skip_counter(R_THRESH);
64      check("R->G", uut.state == G);
65      check("out=10001", out == OUT_G);
66
67      // T3: Async reset from Y and R
68      $display("\n--- T3: Async reset ---");
69      request = 0; wait_clks(2); request = 1;
70      check("in Y", uut.state == Y);
71      reset = 0; wait_clks(2);
72      check("Y reset->G", uut.state == G);
73      reset = 1; wait_clks(2);
74      // Now reset from R
75      request = 0; wait_clks(2); request = 1; wait_clks(1);
76      skip_counter(Y_THRESH);
77      check("in R", uut.state == R);
78      reset = 0; wait_clks(2);
79      check("R reset->G", uut.state == G);
80      reset = 1; wait_clks(2);
81
82      // T4: Idle stability
83      $display("\n--- T4: Idle ---");
84      request = 1; wait_clks(100);
85      check("stays G", uut.state == G);
86
```

```
87      // Summary
88      $display("\n== %0d passed, %0d failed ==", pass_count, fail_count);
89      if (fail_count == 0) $display("ALL TESTS PASSED");
90      else $display("SOME TESTS FAILED");
91      $finish;
92    end
93
94  endmodule
```